

Dimostrazione di proprietà di funzioni sulle liste

Luca Padovani

Linguaggi e Paradigmi di Programmazione

È proibito condividere e divulgare in qualsiasi forma i materiali didattici caricati sulla piattaforma e le lezioni svolte in videoconferenza. Ogni azione che viola questa norma sarà denunciata agli organi di Ateneo e perseguita a termini di legge.

principio di induzione sulle liste finite

- ▶ **ogni** lista finita è costruita a partire dalla lista vuota `[]` e un numero **finito** di applicazioni del costruttore `:` (`cons`)

principio di induzione sulle liste finite

Data una proprietà $P(xs)$ delle liste, se

- ▶ $P([])$ e
- ▶ $P(xs)$ implica $P(x : xs)$ per ogni x e xs ,

allora $P(xs)$ per ogni lista finita xs .

- ▶ come nel caso dell'induzione sui numeri naturali il principio si può generalizzare a tutte le liste **più corte** di quella considerata per dimostrare il caso induttivo

alcune funzioni notevoli sulle liste

```
length :: [a] → Int
length []          = 0
length (_ : xs) = 1 + length xs
```

```
(++) :: [a] → [a] → [a]
(++) []          ys = ys
(++) (x : xs) ys = x : (++) xs ys
```

```
reverse :: [a] → [a]
reverse []          = []
reverse (x : xs) = reverse xs ++ [x]
```

Seguono alcune proprietà che vorremmo dimostrare

- 1 $\forall xs, ys : \text{length } (xs ++ ys) = \text{length } xs + \text{length } ys$
- 2 $\forall xs, ys, zs : xs ++ (ys ++ zs) = (xs ++ ys) ++ zs$
- 3 $\forall xs, ys : \text{reverse } (xs ++ ys) = \text{reverse } ys ++ \text{reverse } xs$

$$P(xs) \stackrel{\text{def}}{=} \text{length } (xs ++ ys) = \text{length } xs + \text{length } ys$$

$$P([]) \quad \text{lato sinistro}$$

$$\begin{aligned} \text{length } ([] ++ ys) \\ &= \text{length } ys \end{aligned} \quad (++) . 1$$

$$P([]) \quad \text{lato destro}$$

$$\begin{aligned} \text{length } [] + \text{length } ys \\ &= 0 + \text{length } ys \\ &= \text{length } ys \end{aligned} \quad \begin{aligned} &(\text{length} . 1) \\ &(\text{proprietà di } +) \end{aligned}$$

$$P(xs) \Rightarrow P(x : xs) \quad \text{lato sinistro}$$

$$\begin{aligned} \text{length } ((x : xs) ++ ys) \\ &= \text{length } (x : (xs ++ ys)) \\ &= 1 + \text{length } (xs ++ ys) \\ &= 1 + \text{length } xs + \text{length } ys \end{aligned} \quad \begin{aligned} & (++) . 2 \\ & (\text{length} . 2) \\ & (\text{ipotesi induttiva}) \end{aligned}$$

$$P(xs) \Rightarrow P(x : xs) \quad \text{lato destro}$$

$$\begin{aligned} \text{length } (x : xs) + \text{length } ys \\ &= (1 + \text{length } xs) + \text{length } ys \\ &= 1 + \text{length } xs + \text{length } ys \end{aligned} \quad \begin{aligned} & (\text{length} . 1) \\ & (\text{proprietà di } +) \end{aligned}$$

$$P(xs) \stackrel{\text{def}}{=} \forall ys, zs : xs ++ (ys ++ zs) = (xs ++ ys) ++ zs$$

$$P([])$$

$$[] ++ (ys ++ zs)$$

$$= ys ++ zs \quad (++) . 1$$

$$= ([] ++ ys) ++ zs \quad (++) . 1$$

$$P(xs) \Rightarrow P(x : xs)$$

lato sinistro

$$(x : xs) ++ (ys ++ zs)$$

$$= x : (xs ++ (ys ++ zs)) \quad (++) . 2$$

$$= x : ((xs ++ ys) ++ zs) \quad (\text{ipotesi induttiva})$$

$$P(xs) \Rightarrow P(x : xs)$$

lato destro

$$((x : xs) ++ ys) ++ zs$$

$$= (x : (xs ++ ys)) ++ zs \quad (++) . 2$$

$$= x : ((xs ++ ys) ++ zs) \quad (++) . 2$$

$P(xs) \stackrel{\text{def}}{=} \forall ys : \text{reverse } (xs ++ ys) = \text{reverse } ys ++ \text{reverse } xs$

$P([])$ lato sinistro

$\text{reverse } ([] ++ ys)$
 $= \text{reverse } ys$ (++ .1)

$P([])$ lato destro

$\text{reverse } ys ++ \text{reverse } []$
 $= \text{reverse } ys ++ []$ (reverse .1)
 $= \text{reverse } ys$ (proprietà di ++ da dimostrare!)

$P(x : xs)$ lato sinistro

$\text{reverse } ((x : xs) ++ ys)$
 $= \text{reverse } (x : (xs ++ ys))$ (++ .2)
 $= \text{reverse } (xs ++ ys) ++ [x]$ (reverse .2)
 $= (\text{reverse } ys ++ \text{reverse } xs) ++ [x]$ (ipotesi induttiva)

$P(x : xs)$ lato destro

$\text{reverse } ys ++ \text{reverse } (x : xs)$
 $= \text{reverse } ys ++ (\text{reverse } xs ++ [x])$ (reverse .2)
 $= (\text{reverse } ys ++ \text{reverse } xs) ++ [x]$ (associatività di ++)

reverse efficiente

```
rev :: [a] → [a]
rev xs = shunt xs []
  where
    shunt []      ys = ys
    shunt (x : xs) ys = shunt xs (x : ys)
```

rev è corretta?

► $\forall xs : rev\ xs = reverse\ xs$

$P(xs) \stackrel{\text{def}}{=} \text{rev } xs = \text{reverse } xs$

(primo tentativo)

$P([])$

$\text{rev } []$
 $= \text{shunt } [] []$ (rev.1)
 $= []$ (shunt.1)
 $= \text{reverse } []$ (reverse.1)

$P(x : xs)$

lato sinistro

$\text{rev } (x : xs)$
 $= \text{shunt } (x : xs) []$ (rev.1)
 $= \text{shunt } xs [x]$ (shunt.2)

$P(x : xs)$

lato destro

$\text{reverse } (x : xs)$
 $= \text{reverse } xs ++ [x]$ (reverse.2)
 $= \text{rev } xs ++ [x]$ (ipotesi induttiva)
 $= \text{shunt } xs [] ++ [x]$ (rev.1)

► non si riesce a procedere, occorre un risultato più generale

$P(xs) \stackrel{\text{def}}{=} \forall ys : \text{shunt } xs \text{ } ys = \text{reverse } xs ++ ys$

$P([])$

$\text{shunt } [] \text{ } ys$

$= ys$ (shunt.1)

$= [] ++ ys$ (append.1)

$= \text{reverse } [] ++ ys$ (reverse.1)

$P(x : xs)$

lato sinistro

$\text{shunt } (x : xs) \text{ } ys$

$= \text{shunt } xs \text{ } (x : ys)$ (shunt.2)

$= \text{reverse } xs ++ (x : ys)$ (ipotesi induttiva)

$P(x : xs)$

lato destro

$\text{reverse } (x : xs) ++ ys$

$= (\text{reverse } xs ++ [x]) ++ ys$ (reverse.2)

$= \text{reverse } xs ++ ([x] ++ ys)$ (associatività di ++)

$= \text{reverse } xs ++ (x : ([] ++ ys))$ (append.2)

$= \text{reverse } xs ++ (x : ys)$ (append.1)

correttezza di rev

`rev xs`

`= shunt xs []`

`= reverse xs ++ []`

`= reverse xs`

(**rev.1**)

(dimostrato in precedenza)

(proprietà di ++, vedi esercizi)

approccio top-down alla dimostrazione di proprietà

```
filter :: (a → Bool) → [a] → [a]
filter _ [] = []
filter f (x : xs) | f x      = x : filter f xs
                  | otherwise = filter f xs
```

Problema

- ▶ Dimostrare la seguente proprietà di `filter`

$$\forall f, xs : \text{filter } f (\text{reverse } xs) = \text{reverse } (\text{filter } f xs)$$

- ▶ Non è facile immaginare quali proprietà ausiliarie ci serviranno
- ▶ Iniziamo la dimostrazione e annotiamo i **lemmi** che servono
- ▶ Successivamente dimostriamo i lemmi

$P(xs) \stackrel{\text{def}}{=} \forall f : \text{filter } f (\text{reverse } xs) = \text{reverse } (\text{filter } f \ xs)$

$P([])$

$\text{filter } f (\text{reverse } [])$
= $\text{filter } f []$ (reverse.1)
= $[]$ (filter.1)
= $\text{reverse } []$ (reverse.1)
= $\text{reverse } (\text{filter } f [])$ (filter.1)

$P(xs) \Rightarrow P(x : xs)$

$\text{filter } f (\text{reverse } (x : xs))$
= $\text{filter } f (\text{reverse } xs ++ [x])$ (reverse.2)
= $\text{filter } f (\text{reverse } xs) ++ \text{filter } f [x]$ (lemma 1)
= $\text{reverse } (\text{filter } f xs) ++ \text{filter } f [x]$ (ipotesi induttiva)
= $\text{reverse } (\text{filter } f xs) ++ \text{reverse } (\text{filter } f [x])$ (lemma 2)
= $\text{reverse } (\text{filter } f [x] ++ \text{filter } f xs)$ (dimostrato in precedenza)
= $\text{reverse } (\text{filter } f ([x] ++ xs))$ (lemma 1)
= $\text{reverse } (\text{filter } f (x : ([] ++ xs)))$ (append.2)
= $\text{reverse } (\text{filter } f (x : xs))$ (append.1)

esercizi

Dimostrare le seguenti proprietà

- 1 $\forall xs : xs ++ [] = xs$
- 2 $\forall xs : \text{length} (\text{reverse } xs) = \text{length } xs$
- 3 $\forall xs : \text{reverse} (\text{reverse } xs) = xs$
- 4 $\forall xs : \text{sum} (\text{reverse } xs) = \text{sum } xs$
- 5 $\forall xs, ys : \text{sum} (xs ++ ys) = \text{sum } xs + \text{sum } ys$

```
sublist :: Eq a => [a] -> [a] -> Bool
sublist [] _ = True
sublist _ [] = False
sublist (x : xs) (y : ys) | x == y = sublist xs ys
sublist xs (_ : ys) = sublist xs ys
```

- 6 $\forall f, xs : \text{sublist} (\text{filter } f \text{ } xs) \text{ } xs = \text{True}$

principio di induzione su tipi algebrici

```
data Tree a = Leaf | Branch a (Tree a) (Tree a)
```

principio di induzione sugli alberi

Data una proprietà $P(t)$ degli alberi, se

- ▶ $P(\text{Leaf})$ e
- ▶ $P(t_1) \wedge P(t_2)$ implica $P(\text{Branch } x \ t_1 \ t_2)$ per ogni x, t_1 e t_2

allora $P(t)$ per ogni albero (finito) t .

```
leaves :: Tree a → Int
```

```
leaves Leaf = 1
```

```
leaves (Branch _ t1 t2) = leaves t1 + leaves t2
```

```
branches :: Tree a → Int
```

```
branches Leaf = 0
```

```
branches (Branch _ t1 t2) = 1 + branches t1 + branches t2
```

- ▶ $\forall t : \text{leaves } t = 1 + \text{branches } t$

$$P(t) = \text{leaves } t = 1 + \text{branches } t$$

$$P(\text{Leaf})$$

$$\text{leaves Leaf}$$

$$= 1$$

(leaves.1)

$$= 1 + 0$$

(proprietà di +)

$$= 1 + \text{branches Leaf}$$

(branches.1)

$$P(t_1) \wedge P(t_2) \Rightarrow P(\text{Branch } x \ t_1 \ t_2)$$

$$\text{leaves } (\text{Branch } x \ t_1 \ t_2)$$

$$= \text{leaves } t_1 + \text{leaves } t_2$$

(leaves.2)

$$= (1 + \text{branches } t_1) + (1 + \text{branches } t_2)$$

(ipotesi induttiva)

$$= 1 + (1 + \text{branches } t_1 + \text{branches } t_2)$$

(proprietà di +)

$$= 1 + \text{branches } (\text{Branch } x \ t_1 \ t_2)$$

(branches.2)

```
depth :: Tree a → Int
depth Leaf = 0
depth (Branch _ t1 t2) = 1 + max (depth t1) (depth t2)
```

```
elements :: Tree a → [a]
elements Leaf = []
elements (Branch x t1 t2) = x:(elements t1 ++ elements t2)
```

- 1 $\forall t : \text{depth } t \leq \text{leaves } t$
- 2 $\forall t : \text{length } (\text{elements } t) = \text{branches } t$
- 3 $\forall t : \text{branches } t \leq 2^{\text{depth } t} - 1$